

# WebSockets

---

Material de Estudo Completo  
com exemplos práticos em Java (Spring Boot) e Python (FastAPI)

WebSockets

Java + Spring Boot

Python + FastAPI

Nível Intermediário · Abril de 2026

---

## Sumário

1. Introdução aos WebSockets
2. Como funciona o protocolo
3. WebSocket vs HTTP/Polling/SSE
4. Servidor em Java (Spring Boot) — exemplos
5. Cliente em Java
6. Servidor em Python (FastAPI) — exemplos
7. Cliente em Python
8. Interoperabilidade Java ↔ Python
9. Boas práticas, segurança e produção
10. Tratamento de erros e reconexão
11. Exercícios práticos
12. Referências e leitura adicional

# 1. Introdução aos WebSockets

O **WebSocket** é um protocolo de comunicação full-duplex sobre uma única conexão TCP, padronizado pela [RFC 6455](#) em 2011. Diferentemente do HTTP tradicional, que segue um modelo estrito de requisição-resposta, o WebSocket permite que *cliente e servidor enviem mensagens livremente a qualquer momento*, com baixa latência e overhead reduzido.

## 1.1 Por que WebSockets existem?

Antes do WebSocket, aplicações que precisavam de comunicação em tempo real (chats, cotações financeiras, jogos, dashboards) usavam técnicas como:

- **Polling**: o cliente requisitava o servidor a intervalos regulares (ex.: "tem novidade?" a cada 2s).
- **Long polling**: o servidor segurava a requisição até ter algo novo, então respondia.
- **Comet / streaming**: técnicas mais avançadas, mas frágeis e dependentes de implementações proprietárias.

Todas essas abordagens são ineficientes, geram tráfego desnecessário e impõem latência. O WebSocket resolve isso fornecendo um canal persistente e bidirecional após um único *handshake* HTTP.

## 1.2 Casos de uso típicos

| Cenário                          | Por que WebSocket?                                                    |
|----------------------------------|-----------------------------------------------------------------------|
| Chat em tempo real               | Mensagens instantâneas em ambas as direções, com baixíssima latência. |
| Notificações push                | O servidor empurra eventos sem o cliente precisar perguntar.          |
| Dashboards/monitoramento ao vivo | Métricas atualizadas em milissegundos.                                |
| Jogos multiplayer                | Estado compartilhado de baixa latência.                               |
| Colaboração (editores, quadros)  | Sincronização de estado em tempo real entre usuários.                 |
| Cotações financeiras / IoT       | Streams contínuos de dados.                                           |



## 2.3 Ciclo de vida da conexão

```

CLIENTE                                     SERVIDOR
|                                           |
|----- HTTP Upgrade Request ----->|
|<--- 101 Switching Protocols -----|
|   ===== conexão WS ativa =====   |
|<---- frame texto/binário (em qualquer dir) -|
|----- frame texto/binário ----->|
|<---- ping -----|
|----- pong ----->|
|----- close (1000 normal) ----->|
|<---- close -----|
|   ===== conexão fechada =====   |

```

## 2.4 Códigos de fechamento

| Código    | Significado                                       |
|-----------|---------------------------------------------------|
| 1000      | Fechamento normal                                 |
| 1001      | Endpoint indo embora (ex.: aba fechada)           |
| 1002      | Erro de protocolo                                 |
| 1003      | Tipo de dado não aceitável                        |
| 1006      | Anormal (sem frame de close — fechamento abrupto) |
| 1008      | Violação de política                              |
| 1011      | Erro interno do servidor                          |
| 4000–4999 | Códigos privados da aplicação                     |

### 3. WebSocket vs HTTP/Polling/SSE

| Característica  | HTTP/REST            | Long Polling             | SSE                    | WebSocket               |
|-----------------|----------------------|--------------------------|------------------------|-------------------------|
| Direção         | Cliente → Servidor   | Cliente → Servidor       | Servidor → Cliente     | Bidirecional            |
| Conexão         | Curta                | Aberta até resposta      | Persistente            | Persistente             |
| Overhead        | Headers em cada req. | Alto                     | Médio                  | Mínimo (após handshake) |
| Suporte browser | Universal            | Universal                | Bom                    | Excelente               |
| Reconexão       | Não aplicável        | Manual                   | Automática             | Manual                  |
| Binário         | Sim                  | Sim                      | Não (apenas texto)     | Sim                     |
| Caso ideal      | CRUD                 | Notificações esporádicas | Streams unidirecionais | Tempo real bidirecional |

**Regra prática:** use WebSocket quando precisar de comunicação *bidirecional* de baixa latência. Para streams apenas servidor → cliente, considere SSE (mais simples, reconexão automática). Para CRUD, mantenha REST.

## 4. Servidor em Java com Spring Boot

O Spring Framework oferece duas APIs principais para WebSockets: a **API de baixo nível** (`WebSocketHandler`) e o **STOMP sobre WebSocket**, que adiciona um sub-protocolo de mensageria com tópicos e brokers.

### 4.1 Setup do projeto

#### POM.XML — DEPENDÊNCIAS ESSENCIAIS

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-websocket</artifactId>
  </dependency>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
  </dependency>
</dependencies>
```

### 4.2 Exemplo 1 — Echo WebSocket simples

O servidor mais elementar: tudo que recebe, devolve igual.

#### ECHOHANDLER.JAVA

```
package com.example.ws;

import org.springframework.stereotype.Component;
import org.springframework.web.socket.TextMessage;
import org.springframework.web.socket.WebSocketSession;
import org.springframework.web.socket.handler.TextWebSocketHandler;

@Component
public class EchoHandler extends TextWebSocketHandler {

    @Override
    public void afterConnectionEstablished(WebSocketSession session) throws Exception {
        System.out.println("Conexão aberta: " + session.getId());
        session.sendMessage(new TextMessage("Bem-vindo! Sou o servidor de echo."));
    }

    @Override
    protected void handleTextMessage(WebSocketSession session, TextMessage message) throws Exception {
        String payload = message.getPayload();
        System.out.println("Recebido: " + payload);
        session.sendMessage(new TextMessage("Echo: " + payload));
    }

    @Override
    public void afterConnectionClosed(WebSocketSession session,
                                     org.springframework.web.socket.CloseStatus status) {
        System.out.println("Conexão fechada: " + session.getId() + " status=" + status);
    }
}
```

#### WEBSOCKETCONFIG.JAVA

```

package com.example.ws;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.socket.config.annotation.EnableWebSocket;
import org.springframework.web.socket.config.annotation.WebSocketConfigurer;
import org.springframework.web.socket.config.annotation.WebSocketHandlerRegistry;

@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {

    private final EchoHandler echoHandler;

    public WebSocketConfig(EchoHandler echoHandler) {
        this.echoHandler = echoHandler;
    }

    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(echoHandler, "/ws/echo")
            .setAllowedOriginPatterns("*"); // CORS em dev
    }
}

```

Para testar rapidamente no navegador (cole no DevTools):

#### CONSOLE DO NAVEGADOR

```

const ws = new WebSocket("ws://localhost:8080/ws/echo");
ws.onopen = () => ws.send("Olá!");
ws.onmessage = e => console.log("Servidor:", e.data);
ws.onclose = e => console.log("Fechado", e.code);

```

## 4.3 Exemplo 2 — Chat com broadcast

---

Agora vamos manter uma lista de sessões abertas e reenviar cada mensagem para todos.

#### CHATHANDLER.JAVA

```

package com.example.ws;

import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.stereotype.Component;
import org.springframework.web.socket.*;
import org.springframework.web.socket.handler.TextWebSocketHandler;

import java.time.Instant;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

@Component
public class ChatHandler extends TextWebSocketHandler {

    private final Map<String, WebSocketSession> sessions = new ConcurrentHashMap<>();
    private final ObjectMapper json = new ObjectMapper();

    @Override
    public void afterConnectionEstablished(WebSocketSession session) throws Exception {
        sessions.put(session.getId(), session);
        broadcast(Map.of(
            "type", "system",
            "message", "Usuário " + session.getId() + " entrou.",
            "ts", Instant.now().toString()
        ));
    }

    @Override
    protected void handleTextMessage(WebSocketSession session, TextMessage message) throws Exception {
        // Aceita JSON {"user":"alice","text":"oi"}
        Map<String, Object> incoming = json.readValue(message.getPayload(), Map.class);
        broadcast(Map.of(
            "type", "chat",
            "user", incoming.getOrDefault("user", "anon"),
            "text", incoming.getOrDefault("text", ""),
            "ts", Instant.now().toString()
        ));
    }

    @Override
    public void afterConnectionClosed(WebSocketSession session, CloseStatus status) throws Exception {
        sessions.remove(session.getId());
        broadcast(Map.of(
            "type", "system",
            "message", "Usuário " + session.getId() + " saiu.",
            "ts", Instant.now().toString()
        ));
    }

    private void broadcast(Map<String, Object> payload) throws Exception {
        TextMessage msg = new TextMessage(json.writeValueAsString(payload));
        for (WebSocketSession s : sessions.values()) {
            if (s.isOpen()) {
                synchronized (s) { s.sendMessage(msg); }
            }
        }
    }
}

```

**Atenção à concorrência:** `WebSocketSession.sendMessage` não é thread-safe. Em ambientes multi-thread, sincronize a sessão antes de enviar, ou envolva-a com `ConcurrentWebSocketSessionDecorator`.

#### REGISTRANDO O /WS/CHAT

```

// Em WebSocketConfig
registry.addHandler(chatHandler, "/ws/chat")
    .setAllowedOriginPatterns("*");

```

## 4.4 Exemplo 3 — STOMP (pub/sub) sobre WebSocket

Para aplicações maiores, STOMP simplifica o roteamento por *tópicos*. O Spring traz um broker simples embutido (in-memory) e suporte para RabbitMQ/ActiveMQ no nível do `StompBrokerRelay`.

## STOMPCONFIG.JAVA

```
package com.example.ws;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.web.socket.config.annotation.*;

@Configuration
@EnableWebSocketMessageBroker
public class StompConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic", "/queue"); // canais de saída
        config.setApplicationDestinationPrefixes("/app"); // canal de entrada
        config.setUserDestinationPrefix("/user"); // mensagens 1:1
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws/stomp")
            .setAllowedOriginPatterns("*")
            .withSockJS(); // fallback opcional
    }
}
```

## CHATSTOMPCONTROLLER.JAVA

```
package com.example.ws;

import org.springframework.messaging.handler.annotation.*;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.stereotype.Controller;

import java.time.Instant;
import java.util.Map;

@Controller
public class ChatStompController {

    private final SimpMessagingTemplate broker;
    public ChatStompController(SimpMessagingTemplate broker) { this.broker = broker; }

    // Cliente envia para /app/chat -> servidor publica em /topic/chat
    @MessageMapping("/chat")
    @SendTo("/topic/chat")
    public Map<String, Object> onMessage(Map<String, Object> msg) {
        return Map.of(
            "user", msg.getOrDefault("user", "anon"),
            "text", msg.getOrDefault("text", ""),
            "ts", Instant.now().toString()
        );
    }

    // Mensagem direta para um usuário específico
    public void sendPrivate(String username, String text) {
        broker.convertAndSendToUser(username, "/queue/private",
            Map.of("text", text, "ts", Instant.now().toString()));
    }
}
```

Cliente em JavaScript usando `@stomp/stompjs` :

## CLIENTE STOMP NO BROWSER

```
import { Client } from "@stomp/stompjs";

const client = new Client({
  brokerURL: "ws://localhost:8080/ws/stomp",
  reconnectDelay: 2000,
  onConnect: () => {
    client.subscribe("/topic/chat", m => {
      const data = JSON.parse(m.body);
      console.log(` ${data.user}: ${data.text}`);
    });
    client.publish({
      destination: "/app/chat",
      body: JSON.stringify({ user: "alice", text: "oi pessoal!" })
    });
  }
});
client.activate();
```

## 4.5 Exemplo 4 — Notificações com autenticação JWT

Cenário: enviar notificações ao usuário logado quando algo importante acontece. Vamos validar um JWT no handshake.

JWTHANDSHAKEINTERCEPTOR.JAVA

```
package com.example.ws;

import org.springframework.http.server.*;
import org.springframework.web.socket.WebSocketHandler;
import org.springframework.web.socket.server.HandshakeInterceptor;

import java.util.Map;

public class JwtHandshakeInterceptor implements HandshakeInterceptor {

  @Override
  public boolean beforeHandshake(ServerHttpRequest request, ServerHttpResponse response,
    WebSocketHandler wsHandler, Map<String, Object> attrs) {
    String token = request.getURI().getQuery(); // ex.: ?token=xxx
    if (token == null || !token.startsWith("token=")) return false;
    String jwt = token.substring("token=".length());
    try {
      String username = JwtUtil.validateAndGetSubject(jwt); // sua util
      attrs.put("username", username);
      return true;
    } catch (Exception e) {
      response.setStatusCode(org.springframework.http.HttpStatus.UNAUTHORIZED);
      return false;
    }
  }

  @Override
  public void afterHandshake(ServerHttpRequest req, ServerHttpResponse res,
    WebSocketHandler h, Exception ex) { }
}
```

NOTIFICATIONHANDLER.JAVA

```

@Component
public class NotificationHandler extends TextWebSocketHandler {

    private final Map<String, WebSocketSession> byUser = new ConcurrentHashMap<>();

    @Override
    public void afterConnectionEstablished(WebSocketSession session) {
        String user = (String) session.getAttributes().get("username");
        byUser.put(user, session);
    }

    public void notify(String user, String text) throws IOException {
        WebSocketSession s = byUser.get(user);
        if (s != null && s.isOpen()) {
            s.sendMessage(new TextMessage(
                "{\"type\":\"notification\",\"text\":\"" + text + "\"}"));
        }
    }

    @Override
    public void afterConnectionClosed(WebSocketSession session, CloseStatus status) {
        String user = (String) session.getAttributes().get("username");
        byUser.remove(user);
    }
}

// E no WebSocketConfig:
registry.addHandler(notificationHandler, "/ws/notifications")
    .addInterceptors(new JwtHandshakeInterceptor())
    .setAllowedOriginPatterns("*");

```

## 4.6 Exemplo 5 — Streaming de cotações em tempo real

Servidor emite preços atualizados a cada 500ms para todos os assinantes.

QUOTESTREAMHANDLER.JAVA

```

@Component
public class QuoteStreamHandler extends TextWebSocketHandler {

    private final Set<WebSocketSession> subscribers = ConcurrentHashMap.newKeySet();
    private final ScheduledExecutorService scheduler = Executors.newSingleThreadScheduledExecutor();
    private final Random rnd = new Random();
    private double price = 100.0;

    @PostConstruct
    public void start() {
        scheduler.scheduleAtFixedRate(this::tick, 0, 500, TimeUnit.MILLISECONDS);
    }

    private void tick() {
        price += (rnd.nextDouble() - 0.5) * 2; // random walk
        String msg = String.format("{\"symbol\":\"PETR4\",\"price\":%.2f,\"ts\":%d}",
            price, System.currentTimeMillis());
        for (WebSocketSession s : subscribers) {
            try {
                if (s.isOpen()) synchronized (s) { s.sendMessage(new TextMessage(msg)); }
            } catch (IOException ignored) { }
        }
    }

    @Override public void afterConnectionEstablished(WebSocketSession s) { subscribers.add(s); }
    @Override public void afterConnectionClosed(WebSocketSession s, CloseStatus st) { subscribers.remove(s); }
}

```

## 5. Cliente WebSocket em Java

---

O Java 11+ traz cliente nativo via `java.net.http.WebSocket`. Não precisa de dependência extra.

ECHOCLIENT.JAVA

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.WebSocket;
import java.util.concurrent.CompletionStage;
import java.util.concurrent.TimeUnit;

public class EchoClient {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();
        WebSocket ws = client.newWebSocketBuilder()
            .buildAsync(URI.create("ws://localhost:8080/ws/echo"), new WebSocket.Listener() {
                @Override public void onOpen(WebSocket ws) {
                    System.out.println("Conectado");
                    ws.sendText("Olá Spring!", true);
                    WebSocket.Listener.super.onOpen(ws);
                }
                @Override public CompletionStage<?> onText(WebSocket ws, CharSequence data, boolean last) {
                    System.out.println("Recebido: " + data);
                    return WebSocket.Listener.super.onText(ws, data, last);
                }
                @Override public CompletionStage<?> onClose(WebSocket ws, int code, String reason) {
                    System.out.println("Fechado " + code + " " + reason);
                    return null;
                }
                @Override public void onError(WebSocket ws, Throwable err) { err.printStackTrace(); }
            })
            .get(5, TimeUnit.SECONDS);

        // Mantém vivo
        Thread.sleep(3000);
        ws.sendClose(WebSocket.NORMAL_CLOSURE, "tchau").get();
    }
}
```

## 6. Servidor em Python com FastAPI

---

FastAPI tem suporte nativo e idiomático para WebSockets, baseado em Starlette + asyncio.

### 6.1 Setup

---

#### INSTALAÇÃO

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install fastapi "uvicorn[standard]" websockets
```

#### ESTRUTURA

```
app/
├─ main.py
├─ chat.py
├─ notifications.py
└─ quotes.py
```

### 6.2 Exemplo 1 — Echo

---

#### MAIN.PY — ENDPOINT /WS/ECHO

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()

@app.websocket("/ws/echo")
async def echo(ws: WebSocket):
    await ws.accept()
    await ws.send_text("Bem-vindo ao servidor de echo!")
    try:
        while True:
            data = await ws.receive_text()
            await ws.send_text(f"Echo: {data}")
    except WebSocketDisconnect:
        print("Cliente desconectou")
```

#### RODAR

```
uvicorn app.main:app --reload --port 8000
```

### 6.3 Exemplo 2 — Chat broadcast com gerenciador de conexões

---

#### CHAT.PY

```

from fastapi import APIRouter, WebSocket, WebSocketDisconnect
from datetime import datetime, timezone
import json
from typing import Dict

router = APIRouter()

class ConnectionManager:
    def __init__(self) -> None:
        self.active: Dict[str, WebSocket] = {}

    async def connect(self, ws: WebSocket, user: str) -> None:
        await ws.accept()
        self.active[user] = ws

    def disconnect(self, user: str) -> None:
        self.active.pop(user, None)

    async def broadcast(self, payload: dict) -> None:
        text = json.dumps(payload)
        # Snapshot da lista para evitar mutação durante iteração
        for user, ws in list(self.active.items()):
            try:
                await ws.send_text(text)
            except Exception:
                self.disconnect(user)

manager = ConnectionManager()

@router.websocket("/ws/chat/{user}")
async def chat(ws: WebSocket, user: str):
    await manager.connect(ws, user)
    await manager.broadcast({
        "type": "system",
        "message": f"{user} entrou no chat.",
        "ts": datetime.now(timezone.utc).isoformat()
    })
    try:
        while True:
            data = await ws.receive_text()
            await manager.broadcast({
                "type": "chat",
                "user": user,
                "text": data,
                "ts": datetime.now(timezone.utc).isoformat()
            })
    except WebSocketDisconnect:
        manager.disconnect(user)
        await manager.broadcast({
            "type": "system",
            "message": f"{user} saiu do chat.",
            "ts": datetime.now(timezone.utc).isoformat()
        })

```

#### REGISTRANDO O ROTEADOR

```

# main.py
from fastapi import FastAPI
from .chat import router as chat_router

app = FastAPI()
app.include_router(chat_router)

```

## 6.4 Exemplo 3 — Salas (rooms) com mensagens privadas

ROOMS.PY

```

from fastapi import APIRouter, WebSocket, WebSocketDisconnect
from collections import defaultdict
import json

router = APIRouter()

# room -> {user: WebSocket}
rooms: dict[str, dict[str, WebSocket]] = defaultdict(dict)

async def send_to_room(room: str, payload: dict, exclude: str | None = None) -> None:
    text = json.dumps(payload)
    for user, ws in list(rooms[room].items()):
        if user == exclude:
            continue
        try:
            await ws.send_text(text)
        except Exception:
            rooms[room].pop(user, None)

@router.websocket("/ws/rooms/{room}/{user}")
async def room_chat(ws: WebSocket, room: str, user: str):
    await ws.accept()
    rooms[room][user] = ws
    await send_to_room(room, {"type": "join", "user": user})
    try:
        while True:
            raw = await ws.receive_text()
            try:
                msg = json.loads(raw)
            except json.JSONDecodeError:
                await ws.send_text(json.dumps({"error": "JSON inválido"}))
                continue

            if msg.get("type") == "private" and msg.get("to"):
                target = rooms[room].get(msg["to"])
                if target:
                    await target.send_text(json.dumps({
                        "type": "private", "from": user, "text": msg.get("text", "")
                    }))
            else:
                await send_to_room(room, {
                    "type": "chat", "user": user, "text": msg.get("text", "")
                })
    except WebSocketDisconnect:
        rooms[room].pop(user, None)
        await send_to_room(room, {"type": "leave", "user": user})

```

## 6.5 Exemplo 4 — Notificações em tempo real com tarefas em background

Cenário comum: o front-end conectado recebe notificações disparadas por jobs ou pelo próprio backend.

NOTIFICATIONS.PY

```

from fastapi import APIRouter, WebSocket, WebSocketDisconnect, Depends, HTTPException
from typing import Dict
import asyncio
import json

router = APIRouter()
user_sockets: Dict[str, WebSocket] = {}
notification_queue: asyncio.Queue = asyncio.Queue()

# Validação simples – em produção, use OAuth2/JWT
def validate_token(token: str) -> str:
    if not token:
        raise HTTPException(401)
    return token # devolve o "username" derivado do token

@router.websocket("/ws/notifications")
async def notifications(ws: WebSocket, token: str | None = None):
    if not token:
        await ws.close(code=4401) # código privado: não autorizado
        return
    user = validate_token(token)
    await ws.accept()
    user_sockets[user] = ws
    try:
        while True:
            # Mantém a conexão viva e ignora mensagens do cliente
            await ws.receive_text()
    except WebSocketDisconnect:
        user_sockets.pop(user, None)

# REST endpoint que dispara notificação
@router.post("/notify/{user}")
async def trigger(user: str, message: str):
    ws = user_sockets.get(user)
    if not ws:
        return {"status": "user_offline"}
    await ws.send_text(json.dumps({"type": "notification", "text": message}))
    return {"status": "sent"}

# Worker que consome da fila e propaga notificações
async def notification_worker():
    while True:
        item = await notification_queue.get()
        ws = user_sockets.get(item["user"])
        if ws:
            try:
                await ws.send_text(json.dumps(item["payload"]))
            except Exception:
                pass

@router.on_event("startup")
async def start_worker():
    asyncio.create_task(notification_worker())

```

**Em produção** use Redis Pub/Sub ou um broker (RabbitMQ, NATS) no lugar da `asyncio.Queue` em memória, especialmente quando rodar múltiplas instâncias atrás de um load balancer.

## 6.6 Exemplo 5 — Streaming periódico (cotações)

QUOTES.PY

```

from fastapi import APIRouter, WebSocket, WebSocketDisconnect
import asyncio, random, json, time

router = APIRouter()

@router.websocket("/ws/quotes/{symbol}")
async def quotes(ws: WebSocket, symbol: str):
    await ws.accept()
    price = 100.0
    try:
        while True:
            price += (random.random() - 0.5) * 2
            await ws.send_text(json.dumps({
                "symbol": symbol.upper(),
                "price": round(price, 2),
                "ts": int(time.time() * 1000)
            }))
            await asyncio.sleep(0.5)
    except WebSocketDisconnect:
        return

```

## 6.7 Exemplo 6 — Dashboard de métricas em tempo real

Servidor envia métricas de CPU, memória e latência simuladas a cada segundo.

**METRICS.PY**

```

from fastapi import APIRouter, WebSocket, WebSocketDisconnect
import asyncio, json, random

router = APIRouter()
clients: set[WebSocket] = set()

async def producer():
    while True:
        snapshot = {
            "cpu": round(random.uniform(0, 100), 1),
            "mem": round(random.uniform(40, 90), 1),
            "latency_ms": random.randint(10, 250),
        }
        for ws in list(clients):
            try:
                await ws.send_text(json.dumps(snapshot))
            except Exception:
                clients.discard(ws)
        await asyncio.sleep(1)

@router.on_event("startup")
async def start_producer():
    asyncio.create_task(producer())

@router.websocket("/ws/metrics")
async def metrics(ws: WebSocket):
    await ws.accept()
    clients.add(ws)
    try:
        while True:
            await ws.receive_text() # ignora; mantém conexão
    except WebSocketDisconnect:
        clients.discard(ws)

```

## 7. Cliente WebSocket em Python

---

### 7.1 Cliente assíncrono com a biblioteca `websockets`

---

CLIENTE\_ASYNC.PY

```
import asyncio
import websockets
import json

async def main():
    uri = "ws://localhost:8000/ws/chat/alice"
    async with websockets.connect(uri) as ws:
        await ws.send("oi pessoal!")
        try:
            while True:
                msg = await ws.recv()
                print("<<", json.loads(msg))
        except websockets.ConnectionClosed:
            print("Conexão encerrada")

asyncio.run(main())
```

### 7.2 Cliente síncrono (scripts simples)

---

CLIENTE\_SYNC.PY

```
# pip install websocket-client
import websocket

def on_open(ws):
    print("aberto"); ws.send("oi")
def on_message(ws, m):
    print("<<", m)
def on_close(ws, code, reason):
    print("fechado", code, reason)
def on_error(ws, err):
    print("erro:", err)

ws = websocket.WebSocketApp(
    "ws://localhost:8000/ws/echo",
    on_open=on_open, on_message=on_message,
    on_close=on_close, on_error=on_error,
)
ws.run_forever ping_interval=20, ping_timeout=10)
```

### 7.3 Cliente Python para o servidor Java/STOMP

---

STOMP\_CLIENT.PY

```
# pip install stomp.py websockets
import asyncio, json
from websockets import connect

async def stomp_chat():
    async with connect("ws://localhost:8080/ws/stomp") as ws:
        # CONNECT
        await ws.send("CONNECT\naccept-version:1.2\nhost:localhost\n\n\x00")
        print(await ws.recv()) # CONNECTED

        # SUBSCRIBE em /topic/chat
        await ws.send("SUBSCRIBE\nid:sub-1\ndestination:/topic/chat\n\n\x00")

        # SEND
        body = json.dumps({"user": "py-bot", "text": "olá da Python"})
        frame = f"SEND\ndestination:/app/chat\ncontent-type:application/json\n\n{body}\x00"
        await ws.send(frame)

        # Lê mensagens
        try:
            while True:
                msg = await ws.recv()
                print("<<", msg)
        except Exception as e:
            print("encerrou:", e)

asyncio.run(stomp_chat())
```

## 8. Interoperabilidade Java ↔ Python

WebSocket é um padrão — qualquer linguagem fala com qualquer linguagem desde que ambas concordem sobre o *protocolo de mensagens* em cima do canal. Em geral, JSON resolve 95% dos casos.

### 8.1 Exemplo combinado: chat em Java + bot Python

Imagine que o servidor é o `ChatHandler` em Spring Boot da seção 4.3. Vamos colocar um bot em Python que, sempre que alguém digitar `/clima`, responde com a previsão.

`BOT_CLIMA.PY`

```
import asyncio, json
import websockets

URI = "ws://localhost:8080/ws/chat"

async def bot():
    async with websockets.connect(URI) as ws:
        await ws.send(json.dumps({"user": "bot", "text": "Bot online. Use /clima"}))
        async for raw in ws:
            data = json.loads(raw)
            if data.get("type") != "chat":
                continue
            text = data.get("text", "")
            if text.strip().startswith("/clima"):
                resposta = "Hoje em SP: 24°C, parcialmente nublado."
                await ws.send(json.dumps({"user": "bot", "text": resposta}))

asyncio.run(bot())
```

### 8.2 Convenções recomendadas para mensagens

`ESQUEMA (JSON SCHEMA-LIKE)`

```
{
  "type": "chat | system | notification | error",
  "user": "string opcional",
  "text": "string opcional",
  "data": {}, # qualquer payload customizado
  "ts": "ISO-8601",
  "id": "uuid opcional" # útil para acks/idempotência
}
```

Defina e documente esse esquema antes de codar — facilita evolução e debug.

## 9. Boas práticas, segurança e produção

### 9.1 Use TLS (wss://)

Em produção, sempre use `wss://` (WebSocket sobre TLS). Em Spring Boot, basta habilitar HTTPS no `application.yml`; em FastAPI, rode atrás de um reverse proxy (nginx, Traefik, Cloudflare) com TLS encerrado nele.

#### APPLICATION.YML — SPRING + TLS

```
server:
  port: 8443
  ssl:
    enabled: true
    key-store: classpath:keystore.p12
    key-store-password: changeit
    key-store-type: PKCS12
```

#### NGINX — PROXY WEBSOCKET

```
location /ws/ {
  proxy_pass http://127.0.0.1:8000;
  proxy_http_version 1.1;
  proxy_set_header Upgrade $http_upgrade;
  proxy_set_header Connection "upgrade";
  proxy_set_header Host $host;
  proxy_read_timeout 3600s;
}
```

### 9.2 Autenticação

O navegador **não** permite enviar headers customizados ao abrir uma conexão WebSocket. Estratégias comuns:

1. **Token na query string:** `wss://api/ws?token=eyJ...`. Simples, mas o token aparece em logs/proxies.
2. **Cookie de sessão:** ideal se sua aplicação já usa sessão HTTP.
3. **Auth message logo após o connect:** o cliente envia `{"type":"auth","token":"..."}` como primeira mensagem; o servidor só libera após validar.
4. **Subprotocol header:** alguns clientes (incluindo o JS nativo) permitem passar tokens via `Sec-WebSocket-Protocol`.

### 9.3 Heartbeats / keepalive

Conexões longas são derrubadas silenciosamente por NATs, proxies e load balancers ociosos. Configure **ping/pong** a cada 20–30 segundos.

#### SPRING BOOT — HEARTBEAT STOMP

```
config.enableSimpleBroker("/topic")
    .setHeartbeatValue(new long[]{ 10000, 10000 })
    .setTaskScheduler(taskScheduler());
```

#### FASTAPI — PING AUTOMÁTICO COM WEBSOCKETS

```
# biblioteca websockets faz ping/pong a cada 20s por padrão
async with websockets.connect(uri, ping_interval=20, ping_timeout=20) as ws:
    ...
```

## 9.4 Limites e backpressure

---

- Limite tamanho de mensagem (Spring: `setMaxTextMessageBufferSize` ; FastAPI: configure no Uvicorn/ASGI).
- Trate clientes lentos: se o buffer de saída crescer demais, encerre a sessão.
- Use rate limiting por sessão (ex.: máximo de N mensagens/seg).

## 9.5 Escalando horizontalmente

---

Quando você tem múltiplas instâncias do servidor atrás de um LB, mensagens publicadas em uma instância não chegam a clientes conectados em outra. Soluções:

- **Sticky sessions + broker externo:** cliente fica preso a uma instância (LB com IP-hash) e instâncias compartilham eventos via Redis Pub/Sub, RabbitMQ ou Kafka.
- **STOMP relay** (Spring): `config.enableStompBrokerRelay("/topic", "/queue").setRelayHost("rabbitmq")`.
- **Redis Pub/Sub** (FastAPI): cada instância assina os mesmos canais e republica para seus clientes locais.

FASTAPI + REDIS PARA BROADCAST MULTI-INSTÂNCIA

```
# pip install redis
import asyncio, json
import redis.asyncio as redis

r = redis.from_url("redis://localhost")

async def fan_out_from_redis(channel: str, local_clients: set):
    pubsub = r.pubsub()
    await pubsub.subscribe(channel)
    async for msg in pubsub.listen():
        if msg["type"] != "message":
            continue
        for ws in list(local_clients):
            try:
                await ws.send_text(msg["data"].decode())
            except Exception:
                local_clients.discard(ws)

async def publish(channel: str, payload: dict):
    await r.publish(channel, json.dumps(payload))
```

## 9.6 Origem (Origin) e CORS

---

**Não confie no `Origin` sozinho:** ele pode ser forjado fora do browser. Para autenticação, sempre exija um token. Para evitar conexões de páginas externas, valide `Origin` no handshake como uma camada extra.

## 10. Tratamento de erros e reconexão

---

### 10.1 Reconexão exponencial no cliente

---

Toda aplicação de produção precisa de reconexão automática com *backoff* exponencial e *jitter*.

#### JAVASCRIPT — RECONEXÃO ROBUSTA

```
function connect(url, onMessage) {
  let attempt = 0;
  let ws;
  const open = () => {
    ws = new WebSocket(url);
    ws.onopen = () => { attempt = 0; console.log("conectado"); };
    ws.onmessage = e => onMessage(e.data);
    ws.onclose = () => {
      const delay = Math.min(30000, 1000 * 2 ** attempt) + Math.random() * 500;
      attempt++;
      console.log(`reconectando em ${delay.toFixed(0)}ms`);
      setTimeout(open, delay);
    };
    ws.onerror = err => console.error(err);
  };
  open();
  return { send: (m) => ws && ws.readyState === 1 && ws.send(m) };
}
```

#### PYTHON — RECONEXÃO

```
import asyncio, random
import websockets

async def consumer(url):
    delay = 1
    while True:
        try:
            async with websockets.connect(url) as ws:
                delay = 1
                async for msg in ws:
                    print("<<", msg)
        except (websockets.ConnectionClosed, OSError) as e:
            jitter = random.random() * 0.5
            await asyncio.sleep(delay + jitter)
            delay = min(30, delay * 2)
```

### 10.2 Idempotência e mensagens em duplicidade

---

Após reconexões, mensagens podem ser entregues duas vezes (se o cliente confirmou e a confirmação se perdeu). Solução clássica: cada mensagem leva um `id` único e o servidor mantém um conjunto recente de IDs processados, descartando duplicatas.

### 10.3 Logging útil

---

| O que registrar             | Para quê                                                      |
|-----------------------------|---------------------------------------------------------------|
| open / close / error        | Visibilidade operacional                                      |
| código de close             | Distinguir 1000 (normal) de 1006 (rede morrendo) e 4xxx (app) |
| latência ping → pong        | Detectar degradação de rede                                   |
| tamanho de fila por sessão  | Identificar clientes lentos                                   |
| contagem de conexões ativas | Capacity planning                                             |

## 11. Exercícios práticos

### Exercício 1 — Echo com timestamp

Modifique o `EchoHandler` (Java) e o endpoint `/ws/echo` (Python) para que respondam `"Echo (em 12ms): mensagem"`, medindo o tempo entre o recebimento e o envio.

**Pista:** use `System.nanoTime()` em Java e `time.perf_counter()` em Python.

### Exercício 2 — Chat com histórico

Estenda o exemplo de chat para que, ao se conectar, o cliente receba as últimas 50 mensagens. Mantenha um buffer circular no servidor.

**Pistas:**

- Java: `java.util.concurrent.ConcurrentLinkedDeque` com tamanho controlado.
- Python: `collections.deque(maxlen=50)`.

### Exercício 3 — Salas e moderação

A partir de `rooms.py`, adicione comandos administrativos: `/kick <user>`, `/mute <user>`. Apenas o primeiro a entrar na sala deve poder usá-los.

### Exercício 4 — Bot Python no servidor Java

Conecte o `bot_clima.py` ao seu `ChatHandler` Java. O bot deve responder, além de `/clima`:

- `/hora` → hora atual
- `/dado` → um número aleatório de 1 a 6
- `/users` → quantidade de usuários online (você precisará adicionar essa contagem ao broadcast do servidor)

### Exercício 5 — Streaming de logs

Crie um endpoint WebSocket em FastAPI que faça `tail -f` em um arquivo de log e envie as novas linhas em tempo real para o cliente. Trate a desconexão para parar a leitura.

**Pista:** use `aiofiles` e leitura por offset.

### Exercício 6 — Quiz multiplayer

Implemente, em Spring Boot, um quiz em que:

1. O primeiro cliente conectado vira "host" e envia perguntas via `/quiz/question`.
2. Outros clientes recebem a pergunta, respondem em até 10 segundos.
3. O servidor distribui pontuação e ranqueamento ao final de cada rodada.

Para o front-end, use HTML simples ou um cliente Java/Python.

### Exercício 7 — Reconexão com retomada de estado

Implemente o protocolo "resume": o cliente envia, ao conectar, o último `id` recebido; o servidor reenvia somente o que veio depois daquele id. Use o exemplo de cotações como base.

### Exercício 8 — Carga: testando seu servidor

Use a ferramenta `artillery` ou escreva um script Python que abra 1.000 conexões simultâneas no seu servidor e dispare mensagens. Monitore consumo de CPU/memória e identifique gargalos.

**Pista:** em Python, `asyncio.gather` com 1.000 corrotinas `websockets.connect` resolve.

## 12. Referências e leitura adicional

---

### Especificações e padrões

- RFC 6455 — The WebSocket Protocol: <https://datatracker.ietf.org/doc/html/rfc6455>
- RFC 7692 — Compression Extensions for WebSocket
- STOMP 1.2: <https://stomp.github.io/stomp-specification-1.2.html>

### Java / Spring

- Spring WebSocket: [docs.spring.io/spring-framework/reference/web/websocket.html](https://docs.spring.io/spring-framework/reference/web/websocket.html)
- Java EE Tyrus (JSR-356): [projects.eclipse.org/projects/ee4j.tyrus](https://projects.eclipse.org/projects/ee4j.tyrus)
- java.net.http.WebSocket — JDK API: javadoc oficial

### Python / FastAPI

- FastAPI WebSockets: [fastapi.tiangolo.com/advanced/websockets/](https://fastapi.tiangolo.com/advanced/websockets/)
- websockets (asyncio): [websockets.readthedocs.io](https://websockets.readthedocs.io)
- Starlette WebSockets: [starlette.io/websockets/](https://starlette.io/websockets/)

### Ferramentas úteis

- **wscat**: cliente CLI ( `npm i -g wscat` ) — `wscat -c ws://localhost:8080/ws/echo`
- **websocat**: o "netcat" para WebSocket
- **Postman**: tem suporte nativo para conexões WebSocket
- **Chrome DevTools**: aba "Network" → tipo "WS" mostra todos os frames trocados

---

*Material elaborado para estudo — Abril/2026*

Para imprimir como PDF: **Ctrl+P** (Windows/Linux) ou **Cmd+P** (Mac) → "Salvar como PDF"